

DAY 10

FUNCTIONS WITH OUTPUTS

Goals:

- Functions with outputs
- A calculator app
- Today we'd master the concept of functions to escape code redundancy

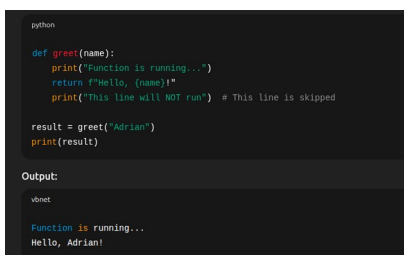
FUNCTIONS WITH OUTPUTS

Recap on Functions:

- The first we learnt was just how to define functions; that taught us how to reduce the amount of code we have to write when we have instructions that need to be executed repeatedly
- The next kind we learnt could take an input inside the parenthesis, this gives the ability to modify the code inside the function and get it to do something different each time, depending on the input passed in.

More on Functions:

- This would be the final flavour on functions, one that allows output
- inputs and outputs are completely separate from each other, you don't necessarily need to have an input to have an output
- the output keyword is **return**, we use the **return** keyword followed by what we want to return (**return result**), when we call this function later and it runs it'd output result (**this doesn't mean return is now a print function, but rather wherever we call the function it'd now be replaced by the return**), and we can save the functions output into another variable
- to format a string to title case you can either use the **"".title()** or, **string.capwords("")** after importing the string module
- The **.title()** method treats any non-letter character (like an apostrophe) as a word boundary. So, it capitalizes letters after these characters, which can lead to awkward results.
 - For example: **"they're bill's friends from the UK".title()**
 - Output: **"They'Re Bill'S Friends From The Uk"**
- On the other hand, **string.capwords()** only splits the sentence by spaces, so it avoids this issue and gives more natural capitalization.
- We can store the output(**return**) of a function inside a variable, or we can print the output of the function directly
- don't be too big to use **thonny**
- Even built-in functions like the **len()** function **returns** an output
- code after a return statement is not executed:

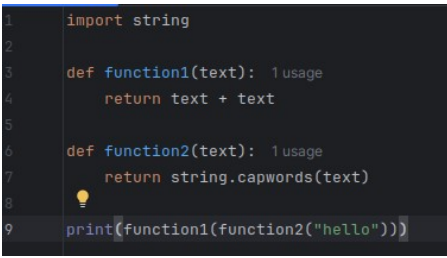


```
python
def greet(name):
    print("Function is running...")
    return "Hello, (name)!"
    print("This line will NOT run") # This line is skipped

result = greet("Adrian")
print(result)

Output:
vbnrt
Function is running...
Hello, Adrian!
```

- to make the output of one function, the input of another we simply use the format;
`function2(function1("hello"))`

- 

```
1 import string
2
3 def function1(text): 1 usage
4     return text + text
5
6 def function2(text): 1 usage
7     return string.capwords(text)
8
9 print(function1(function2("hello")))
```

- output: `HelloHello`

MULTIPLE RETURN VALUES

- no line of code gets executed after the return keyword
- it is possible to use the `input()` function as a keyword argument:

- ```
1 def format_name(f_name, l_name): 1 usage
2 formatted_f_name = f_name.title()
3 formatted_l_name = l_name.title()
4 return f"{formatted_f_name} {formatted_l_name}"
5
6
7 print(format_name(f_name= input("What is your first name: "), l_name=input("What is your last name: ")))
8
```

- it's possible to use an earlier return, to terminate:

- ```
1 def format_name(f_name, l_name): 1 usage
2     if f_name == "" or l_name == "":
3         return "You did not provide valid inputs"
4     formatted_f_name = f_name.title()
5     formatted_l_name = l_name.title()
6     return f"{formatted_f_name} {formatted_l_name}"
7
8
9 print(format_name(f_name= input("What is your first name: "), l_name=input("What is your last name: ")))
10
```

it's always safer to return with a value, it's both proper formatting of code and it helps with better readability

Coding Exercise: Leap Year

- [video explanation](#)
- [flowchart](#)
- you can also return boolean values and use those boolean values in your code
- return is not a print() function, even after your function returns a value, if you do not print the function or store it in a variable and print it, you wouldn't see an output

DOCSTRINGS

- this is used to add documentation to your functions
- to do this, it has to be the immediate line after the definition of the function
- it has to be written inbetween triple quotation marks, and it can also be multiline
- once you have added a docstring to your function, calling the function's documentation will display the text you wrote
- a docstring can also be used as a multiline comment, but it is ill advised because if it's assigned to a variable it becomes code, and it can get confusing down the line.
- By the way, it isn't a comment even when placed randomly in code, it's just ignored by python as it isn't used
- so depending on it's position defines it's use

Quiz

- In python, you can define a function inside a function

```
1 | def outer_function(a, b):  
2 |     def inner_function(c, d):  
3 |         return c + d  
4 |     return inner_function(a, b)  
5 |  
6 | result = outer_function(5, 10)  
7 | print(result)
```

the 4th line calls the function `inner_function()` with the variables defined in the outer function as the input

THE CALCULATOR PROJECT

- [demo](#)
- to use a function without triggering it, all you have to do is type out the function without the parenthesis; this helps you use the function as a value
- if you store a function as a value under a different variable, that variable becomes a function identical to the function before it, you can call the new function (variable) with input as you would have the function you stored as a value
- you can define multiple functions and store them in a dictionary as values, and then loop through them and use their different functionalities, to do this you simply access the function as a value `Dict[key]`
- to give input to a function value of a dictionary; after calling the value, pass the input using keyword arguments in brackets:
 - `print(operations["*"])(n1 = 4, n2= 8))`
- A function shouldn't be limited to a single functionality, if your entire code builds on itself to a single functionality you can also define it as a function, so to use you can simply call it with a single line
- Calling a function within itself is called a recursion, (So yeah — you can think of recursion as a *loop that works by stacking function calls instead of cycling in place*. The big mental shift is that recursion *calls itself* rather than going “back to the top” in the same frame.)
- it's advisable to always have a base case (break case) within your recursive code or it's bound to lead to a recursive error or crash
- always remember, a day's project are not perfect projects, they're training projects, bases for you to build more upon, always feel free to research on related projects and improve functionality